

AX

AXON-X · AXON-WATCH

How-To *Handbook*

Your field guide for bootstrapping, verifying, troubleshooting,
and shipping thin slices in the next-generation operator
console.

Shell : 4173

Control plane : 8787

Watch : 8788

Last verified · 2026-07-04 — `npm run verify` OK, 82
Python tests OK, stack smoke on ports 4173/8787/8788
OK.

Generated 04 Jul 2026

CONSOLE-WEB

CONTROL-PLANE

SHARED-TYPES

16 chapters

1

Terminology And Abbreviations

2

What Axon-X Is

3

What State The Repo Is In Right Now

4

Source Of Truth Rules

5

Repo Layout

6

The Current Architecture In Plain English

7

Quick Start

8

What The Current App Does On Boot

9

Locked Shell Layout

10

Current Shell Layout

11

Mockup / Live Parity Notes

12

The Most Important Files Right Now

13

Verification Commands

14

What PASS / PENDING / FAIL Mean In Verification

15

Common Working Patterns

16

Troubleshooting

17

Tips And Tricks

18

What A Good Next Slice Looks Like

Read this first

Practical guide for operators, reviewers, and developers working in the **axon-watch** repo — the greenfield home of **Axon-X**.

- Plans live in `axon-local/Plans/Axon-Watch/` — implement here, don't redefine semantics there.
- Start with `./scripts/dev/up.sh`, verify with `npm run verify`.
- Treat bootstrap-real and feature-real as different things — see parity ledger before assuming parity.

01 Terminology And Abbreviations

Use this glossary when reading plans, ADRs, code, or agent summaries.

TERM	MEANING
ADR	Architecture Decision Record — a numbered, immutable write-up of a significant technical or process choice. Accepted ADRs live in <code>docs/adr/</code> . Do not rewrite them; supersede with a new ADR instead.
Axon-X	User-facing product name for the next-generation operator console.
axon-watch	Internal repo folder and npm workspace name. Legacy naming; not the product label shown to operators.
axon-local	The current production Axon app repo (port 7734). Planning for Axon-X still lives here under <code>Plans/Axon-Watch/</code> .
Briefing seam	<code>GET /api/briefing</code> returns canonical <code>OperatorBriefing</code> . The shell loads it at bootstrap and projects that data across the right dock: approvals, signals, and the KAIRO briefing card all read from the same briefing/runtime truth. Approval mutations stay on the run approval seam. See <code>docs/contracts/BRIEFING-SEAM.md</code> .
Control plane	FastAPI service on port 8787 that owns run truth, runtime summary, inbox projection, workspaces list, and briefing.
Console-web / shell	Vue 3 frontend on port 4173 — the visible Axon-X UI.
Contract / shared contract	Canonical TypeScript types and JSON fixtures in <code>packages/shared-types/</code> . Frontend and backend must agree here first.
Coordinator lane	Single owner of serial-only semantics during multi-agent work. See <code>docs/MULTITASK-LANES.md</code> .
DTO	Data Transfer Object — a typed payload shape exchanged between services or UI layers (for example <code>RuntimeSummary</code> , <code>RunRecord</code>).
Fitness function	An automated check that guards architecture or performance (dependency direction, DTO size budgets, latency thresholds).
Frozen planning bundle	The locked docs under <code>axon-local/Plans/Axon-Watch/</code> . Implementation must not silently drift from these.

TERM	MEANING
KAIRO	Knowledge-Augmented Intelligence for Response and Oversight — the planned operator-presence layer (watching, advising, interrupting, executing with receipts). The current shell has KAIRO presentation scaffolding (topbar presence module, briefing card, operator copy), but the full operator-presence system in <code>KAIRO_MODE.md</code> / <code>ADR-005-kairo-as-operator-presence-layer.md</code> is not landed yet.
Lane A/B/C/D	Parallel implementation ownership areas defined in <code>docs/MULTITASK-LANES.md</code> (watch, shell, control-plane, dev/verify).
Monaco host	In-browser code editor surface (<code>EditorHost.vue</code>). Loads workspace files on disk (README.md, notes.txt) plus read-only DTO overview tabs.
Parity ledger	Checklist of behaviors Axon-X must eventually match from current Axon. Lives in frozen planning.
Run / run record	Canonical execution unit with <code>phase</code> , <code>status</code> , capability flags, and transition history.
Run phase	High-level lifecycle stage (<code>queued</code> , <code>starting</code> , <code>executing</code> , <code>awaiting_approval</code> , <code>review_ready</code> , <code>completed</code> , etc.). Defined in frozen <code>run-state.md</code> .
Runtime summary	Boot-critical DTO at <code>GET /api/runtime/summary</code> — identity, watch connection, active runs, approvals snapshot.
Signal / inbox item	Watch-produced event surfaced through control-plane <code>GET /api/inbox</code> .
Thin slice	A small, verifiable vertical increment — one owned behavior with tests, not a broad rewrite.
Watch service	FastAPI service on port 8788 that produces canonical signals; control-plane projects them into inbox/runtime summary.
xterm host	In-browser terminal surface (<code>TerminalHost.vue</code>) attached to a backend PTY session via <code>WS /api/workspaces/{workspace_id}/terminal</code> . Runs real shell commands in a workspace-scoped directory under <code>.local/workspaces/</code> (override with <code>AXON_WATCH_WORKSPACE_ROOT</code>).

TERM	MEANING
Workspace	Logical operator context keyed by <code>workspace_id</code> . Today the API returns IDs only; rich catalog metadata is deferred.

Two repos, two apps

Do not confuse these:

	AXON-LOCAL (CURRENT AXON)	AXON-WATCH (AXON-X)
Default URL	<code>http://127.0.0.1:7734</code>	<code>http://127.0.0.1:4173</code>
Start command	<code>./start.sh</code> from axon-local	<code>./scripts/dev/up.sh</code> from axon-watch
Status	Mature daily-driver console	Early greenfield rebuild
Relationship	Source of parity targets and frozen plans	Implementation target for modernization

02 What Axon-X Is

Axon-X is the next-generation Axon console and operator environment.

The code lives in the `axon-watch` repo folder for now. That folder name is legacy/internal; the product name is **Axon-X**.

The target product combines:

- an IDE-style shell
- a control plane
- a dedicated watcher service

The implementation repo is here:

- `/home/edp/axon-nvme/repos/axon-watch`

The frozen planning source-of-truth still lives here:

- `/home/edp/axon-nvme/repos/axon-local/Plans/Axon-Watch/`

Important rule:

- read plans from `axon-local`
- implement in `axon-watch`

Do not casually move back and forth inventing new semantics in both places.

03 What State The Repo Is In Right Now

The repo is in an early but real bootstrap state.

What already exists:

- repo scaffold
- `console-web` shell skeleton
- `control-plane` FastAPI bootstrap service
- `axon-watch` FastAPI bootstrap service
- shared contract package
- verification harness
- health and readiness scripts

What is already real:

- root workspace scripts
- service startup/shutdown flow
- `/api/runtime/summary`, `/api/inbox`, `/api/runs`, and `/api/briefing` routes in the control plane
- shared contract types under `packages/shared-types/`
- shell consumption of canonical runtime summary, inbox, and run DTOs
- thin Monaco and xterm host surfaces in `console-web`

What is still intentionally thin or deferred:

- full signal production and deeper ranking beyond current inbox rule stack
- deep watch summary logic
- performance evidence for all budgets

What is now real in the thin slice (verified 2026-07-04):

- run create/complete/stop/resume lifecycle
- explicit approval boundary (`requires_approval` , approve/reject)
- review-ready entry, completion, and follow-up resume path
- SQLite-backed run persistence (survives control-plane restart)
- operator briefing loaded at bootstrap and rendered in the right dock (`BriefingPanel.vue`)
- two watch-produced inbox signals with multi-factor ranking (severity, recency, unresolved duration via `created_at` , status, action-type, workspace priority)
- workspace list API and shell workspace selector (IDs only)
- the shell is split into `TopBar` , `LeftSidebar` , `CenterWorkbench` , `RightDock` , and `StatusBar` regions with mockup-shell chrome
- Monaco host bound to canonical DTO documents and **workspace files on disk** with a nested explorer tree, lazy file loading, new-file creation, and active-file rename
- backend PTY terminal attachment for the selected workspace (real shell I/O via WebSocket)
- workspace-scoped conversation rehydration: `GET /api/workspaces/{workspace_id}/chat/thread` plus existing thread history read reloads the Conversation seam after page refresh

Workspace IDs (operator vs catalog):

- **Operator shell** uses `MOCKUP_WORKSPACE_IDS` only (`workspace_smoke` , `workspace_recsys` , ...). `mergeMockupWorkspaceCatalog()` trims API extras so `currentWorkspace` is always sidebar-visible.
- **Control-plane catalog** may still list fixture defaults (`workspace_alpha` , `workspace_bootstrap`) and run/inbox IDs for tests and API consumers; the shell does not select those as `currentWorkspace` .

- Bootstrap picks workspace deterministically: active run workspace (when visible) → `workspace_smoke` default → first mockup workspace.

Manual acceptance for reload-safe chat (use `workspace_smoke`):

1. `./scripts/dev/up.sh`
2. Open `http://127.0.0.1:4173`
3. Confirm `workspace_smoke` is selected (or select it)
4. Post a command in the Command seam
5. Hard reload the page
6. Conversation should rehydrate automatically for the same workspace when an active run or default bootstrap applies; if needed, re-select `workspace_smoke`

API-only check (any valid catalog ID):

```
curl -s http://127.0.0.1:8787/api/workspaces/workspace_smoke/chat/thread
curl -s http://127.0.0.1:8787/api/chat/threads/<thread_id>/history
```

What is **not** real yet despite similar-sounding names:

- **Full KAIRO operator presence** — the current shell has visual scaffolding, but spoken alerts, persona settings, and richer operator-presence behavior are still planned in axon-local docs
- **Full parity with axon-local** — intentional; see parity ledger for gaps

So this repo is not a fake mockup, but it is also not feature-complete or a drop-in replacement for the current Axon UI.

04 Source Of Truth Rules

When you are unsure what something should mean, check the planning bundle.

The most important frozen planning docs are:

- `PRODUCT.md`

- ARCHITECTURE.md
- UI_SPEC.md
- UI_COMPOSITION_SPEC.md
- UI_VISUAL_DIRECTION.md
- UI_REFERENCE_ARCHETYPES.md
- run-state.md
- runtime-summary.md
- signal-events.md
- watch-api.md
- control-api.md
- PARITY_LEDGER.md
- FITNESS_FUNCTIONS.md
- TRANSITION_ARCHITECTURE.md
- CONTRACT_TESTING_SPEC.md

Things you must not redefine casually

These are serial-only semantics and should not drift:

- canonical run phases
- runtime summary vocabulary
- signal identity, severity, and status
- approval semantics
- transition seams and rollback rules
- parity acceptance rules

If the plan seems wrong or insufficient:

- do not silently change it
- raise a proposed amendment instead

05 Repo Layout

Top-level structure:

```
axon-watch/  
  apps/  
    console-web/  
  services/  
    control-plane/  
    axon-watch/  
  packages/  
    shared-types/  
  docs/  
  scripts/  
    dev/  
    verify/  
  infra/  
  tests/
```

What each part owns

apps/console-web/

Owns the integrated shell:

- topbar
- left sidebar
- center workbench (including the embedded terminal dock)
- right dock
- status bar

It should consume canonical contracts, not invent backend truth.

services/control-plane/

Owns interactive backend responsibilities such as:

- UI-facing health/readiness
- runtime summary assembly
- later run-state, approvals, and workspace orchestration

services/axon-watch/

Owns watcher responsibilities such as:

- monitoring loops
- watch health/readiness
- later signal production
- later inbox/signal summaries

packages/shared-types/

Owns canonical shared contract types.

This is one of the most important directories in the repo.

If frontend and backend disagree about what a thing means, the fix should start here, not in ad hoc local types.

scripts/dev/

Owns local run helpers:

- start
- stop
- health checks

scripts/verify/

Owns cheap, repeatable verification:

- dependency direction checks
- DTO size checks
- ADR governance checks
- latency-budget check scaffolding

06 The Current Architecture In Plain English

Think of the new app as three cooperating parts:

1. the shell the user sees
2. the control plane that serves user-facing APIs
3. the watcher service that will eventually observe and normalize signals

The current implementation is following a thin-slice path:

- bootstrap the shell and services first
- land canonical contracts early
- add real endpoints against those contracts
- then deepen behavior

This is deliberate.

It avoids:

- giant rewrites
- hidden semantic drift
- UI and backend inventing different meanings

07 Quick Start

1. Go to the repo

```
cd /home/edp/axon-nvme/repos/axon-watch
```

2. Install JavaScript dependencies

```
npm install
```

This is important because root workspace commands now rely on npm workspaces.

3. Review environment defaults

Default values are provided in:

- `.env.example`

If you need custom ports or local paths:

```
cp .env.example .env
```

Then edit `.env` as needed.

4. Start the local stack

```
./scripts/dev/up.sh
```

`up.sh` does not report success until all three services are actually reachable.

This starts:

- `console-web`
- `control-plane`
- `axon-watch`

Important startup contract:

- ports are fixed by `.env` / `.env.example`
- startup fails fast if `4173`, `8787`, or `8788` is already in use
- startup rolls back partial processes if readiness fails
- services stay detached after the launching shell exits

5. Check health

```
./scripts/dev/check-health.sh
```

Expected endpoints:

- console web: `http://127.0.0.1:4173`
- control plane health: `http://127.0.0.1:8787/api/health`

- watch health: `http://127.0.0.1:8788/internal/watch/health`
- briefing: `http://127.0.0.1:8787/api/briefing` (also loaded by the shell at bootstrap)
- runtime summary: `http://127.0.0.1:8787/api/runtime/summary`
- inbox: `http://127.0.0.1:8787/api/inbox`
- runs: `http://127.0.0.1:8787/api/runs`
- workspaces: `http://127.0.0.1:8787/api/workspaces`

`check-health.sh` also probes `/api/workspaces`.

6. Stop the stack

```
./scripts/dev/down.sh
```

08 What The Current App Does On Boot

Right now, the shell boots and loads five control-plane seams in parallel.

That flow looks like this:

1. `apps/console-web/src/main.ts` creates the Vue app
2. the shell store initializes
3. `loadBootstrapData()` is called
4. the frontend fetches `/api/runtime/summary`, `/api/inbox`, and `/api/briefing`
5. workspaces and runs load sequentially; `resolveBootstrapWorkspaceId()` sets `currentWorkspace` (active run workspace when sidebar-visible, else `workspace_smoke`)
6. workspace files and chat thread history load for that workspace
7. the shell renders topbar context, workspace state, editor/terminal workbench, right-dock seams, and status bar truth strips

Important limitations:

- **Operator workspace list** — shell and sidebar share the same `MOCKUP_WORKSPACE_IDS` catalog; catalog-only IDs such as `workspace_alpha` remain API-visible but are not selected in the shell
- **Bootstrap workspace selection** — deterministic via `resolveBootstrapWorkspaceId()` after sequential workspace + run load (no parallel race)
- **Chat rehydration** — scoped per workspace; messages posted under one ID do not appear when another workspace is selected; only the latest thread per workspace is returned
- Operator mode renders the right dock as **Run** → **Approvals** → **Signals** → **Conversation** → **Command** → **KAIRO Briefing** with the briefing card anchored at the bottom of the dock
- briefing data is projected across the dock rather than shown as one raw DTO dump: approvals stay in the approvals seam, top signals stay in the signals seam, and the KAIRO card stays summary/CTA-oriented
- Monaco host loads workspace files on disk and read-only DTO overview tabs
- xterm host attaches to a backend PTY scoped to the selected workspace directory
- inbox ranking uses severity, recency, unresolved duration (`created_at`), status, action-type, and a thin watch-owned workspace priority map

That is okay for this stage.

09 Locked Shell Layout

Locked 2026-07-04 — see `docs/UI_LAYOUT_LOCK.md` and `docs/adr/ADR-004-locked-console-shell-layout.md`.

Do not rearrange shell regions or dock seam order without a superseding ADR.

Frozen planning in `axon-local/Plans/Axon-Watch/UI_COMPOSITION_SPEC.md` is amended to match this geometry.

Five-region grid:

REGION	COMPONENT	NOTES
Top bar	<code>TopBar.vue</code>	identity zone + runtime strip + KAIRO module + mode toggle
Left sidebar	<code>LeftSidebar.vue</code>	workspaces, optional explorer (IDE), status card
Center workbench	<code>CenterWorkbench.vue</code>	Monaco editor + embedded resizable terminal dock
Right dock	<code>RightDock.vue</code>	Run → Approvals → Signals (upper stack), KAIRO Briefing bottom hero
Status bar	<code>StatusBar.vue</code>	HUD runtime strip

KAIRO Briefing height tracks the workbench terminal dock via `--briefing-dock-height`.

10 Current Shell Layout

The locked shell matches the mockup/live screenshots:

- `TopBar` — brand frame, mockup breadcrumb/version context panel, DTO runtime strip, KAIRO presence module, Operator/IDE toggle, settings action
- `LeftSidebar` — workspace list first, workspace status card second, explorer tree only in IDE mode
- `CenterWorkbench` — editor tabbar + breadcrumb + Monaco editor above a resizable bottom terminal/log dock
- `RightDock` — run seam, approvals seam, signals seam, KAIRO briefing card
- `StatusBar` — persistent watch / run phase / signals / operator strip

This is the layout you should compare against the mockup and live screenshots, not the older single-file shell description from earlier thin slices.

11 Mockup / Live Parity Notes

Parity observations from the current mockup and live screenshots:

- the region geometry now largely matches the mockup: topbar, workspace rail, editor-over-terminal workbench, right dock, and bottom status strip
- the **runtime strip**, status bar zones, and workspace status card derive from live shell state / `RuntimeSummary`, but the topbar breadcrumb and runtime version chips are still mockup-style presentation helpers
- the sidebar and shell store share the same operator workspace catalog (`MOCKUP_WORKSPACE_IDS`)
- the dock still uses scaffold seam titles (`RUN SEAM`, `APPROVALS SEAM`, `SIGNALS SEAM`) rather than the final operator-facing copy
- Operator mode renders the right dock as **Run** → **Approvals** → **Signals** → **Conversation** → **Command** → **KAIRO Briefing** with the briefing card anchored at the bottom of the dock
- the workbench terminal dock default height is ~240px (responsive cap 280px) unless the operator resizes it; height persists in session storage when customized

12 The Most Important Files Right Now

If you need to understand the current implementation quickly, read these first:

Shared contracts

- `packages/shared-types/src/index.ts`
- `packages/shared-types/src/run.ts`
- `packages/shared-types/src/runtime.ts`
- `packages/shared-types/src/signals.ts`
- `packages/shared-types/src/control-plane.ts`

- `packages/shared-types/src/watch.ts`

Fixture payloads

- `packages/shared-types/fixtures/runtime-summary.example.json`
- `packages/shared-types/fixtures/watch-summary.example.json`
- `packages/shared-types/fixtures/run-record.example.json`
- `packages/shared-types/fixtures/signal-event.example.json`

Control plane

- `services/control-plane/app/main.py`
- `services/control-plane/app/workspace_catalog.py`
- `services/control-plane/app/runs/service.py`
- `services/control-plane/app/runtime_summary_assembler.py`
- `services/control-plane/app/operator_briefing.py`

Frontend shell

- `apps/console-web/src/main.ts`
- `apps/console-web/src/api/control-plane.ts`
- `apps/console-web/src/stores/shell.ts`
- `apps/console-web/src/App.vue`
- `apps/console-web/src/components/EditorHost.vue`
- `apps/console-web/src/components/TerminalHost.vue`
- `apps/console-web/src/lib/workspace-documents.ts`

Verification

- `scripts/verify/README.md`
- `scripts/verify/verification_config.json`
- `tests/test_shared_contract_fixtures.py`
- `tests/test_control_plane_runtime_summary.py`
- `tests/test_verify_harness.py`

13 Verification Commands

Use these from the repo root:

Shared contract verification

```
npm run verify:shared-types
```

This confirms the shared-types package typechecks.

Contract verification

```
npm run verify:contracts
```

This verifies:

- shared contract package typing
- shared fixture tests
- control-plane run/inbox/runtime-summary/briefing behavior
- watch signal contract alignment

Full current verification bundle

```
npm run verify
```

This runs:

- contract verification
- console-web typecheck, unit test, and production build
- verify harness checks
- DTO size checks using representative fixtures

Frontend checks

```
npm run typecheck -w @axon-watch/console-web  
npm run test -w @axon-watch/console-web  
npm run build -w @axon-watch/console-web
```

Python syntax checks

```
python3 -m py_compile services/control-plane/app/main.py services/control-  
plane/app/runs/service.py services/control-plane/app/domain/run_state.py  
services/control-plane/app/domain/run_transitions.py services/control-  
plane/app/operator_briefing.py
```

Service tests

```
python3 -m unittest discover -s tests
```

14 What PASS / PENDING / FAIL Mean In Verification

The verify harness uses explicit states.

PASS

The supplied evidence met the rule.

Examples:

- DTO payload fits its size budget
- dependency direction rule is respected

PENDING

The check exists, but the slice does not provide enough real implementation or evidence yet.

Examples:

- shell boot timing evidence not supplied yet
- runtime summary latency budget not yet measured
- no numbered ADR yet exists

PENDING is not automatically bad at this stage.

It means the governance has been created ahead of the full implementation.

FAIL

The rule was violated or the harness itself is broken.

This is the one that should stop you.

15 Common Working Patterns

When adding frontend work

Use real shared types from `packages/shared-types/`.

Do not:

- invent local copies of DTOs
- widen semantics casually in component code
- hide backend meaning in placeholder strings

Good pattern:

- define or refine contract in shared-types
- use it in store/API layer

- render it in Vue components

When adding control-plane work

Prefer:

- canonical DTO assembly
- light endpoint handlers
- explicit payload validation

Avoid:

- stuffing frontend-specific assumptions into the backend
- returning huge boot payloads
- redefining contract shapes inline

When adding watch work

Prefer:

- narrow, owned summaries
- canonical signal/event identities
- explicit watch responsibilities

Avoid:

- pulling UI logic into the watch service
- importing control-plane domain semantics directly
- inventing competing signal vocabulary

16 Troubleshooting

Problem: `./scripts/dev/up.sh` fails or the frontend does not start

Check:

1. Did you run `npm install` at repo root?
2. Are ports `4173` , `8787` , and `8788` free?
3. Did `up.sh` print a specific port conflict or readiness failure message?

Current expected path:

- root install: `npm install`
- root startup: `./scripts/dev/up.sh`
- health check: `./scripts/dev/check-health.sh`

If startup fails, inspect:

- `.local/logs/console-web.log`
- `.local/logs/control-plane.log`
- `.local/logs/axon-watch.log`

Problem: `check-health.sh` fails

This usually means one or more services never started.

Steps:

1. stop everything:

```
./scripts/dev/down.sh
```

2. start again:

```
./scripts/dev/up.sh
```

3. inspect logs:

- `.local/logs/control-plane.log`
- `.local/logs/axon-watch.log`
- `.local/logs/console-web.log`

4. re-run:

```
./scripts/dev/check-health.sh
```

Problem: stale pid files block startup

`up.sh` now clears stale pid files automatically before it checks for a live stack.

First try:

```
./scripts/dev/down.sh
```

If that is not enough, inspect:

- `.local/pids/`
- `.local/logs/console-web.log`
- `.local/logs/control-plane.log`
- `.local/logs/axon-watch.log`

If `up.sh` still fails, it usually means one of the configured ports is held by another live process. Free the port or stop the other process before retrying.

Problem: the shell loads but runtime summary is unavailable

Check:

1. Is control-plane running?
2. Does this endpoint work?

```
curl -fsS http://127.0.0.1:8787/api/runtime/summary | python3 -m json.tool
```

3. If running through Vite dev server, is `/api` proxied to control-plane?

Current dev proxy lives in:

- `apps/console-web/vite.config.ts`

You can also override the target with:

- `VITE_CONTROL_PLANE_BASE_URL`

Problem: `npm run verify` shows `PENDING`

That is often expected at this stage.

Examples of currently expected pending checks:

- shell boot readiness
- runtime summary latency
- watch summary latency
- numbered ADR presence

Treat `PENDING` as:

- “scaffold exists, evidence not landed yet”

Treat `FAIL` as:

- “something is wrong right now”

Problem: dependency direction checks fail

Check for forbidden imports across boundaries.

The current verify harness guards things like:

- watch service must not depend on UI internals
- frontend must not depend on watch internals
- future domain-layer cross-imports must stay clean

If a dependency check fails:

1. remove the cross-boundary import
2. move shared meaning into `packages/shared-types/` if appropriate
3. use an adapter or API boundary instead of direct reach-through

Problem: contract tests fail after changing fixtures or DTOs

This usually means one of three things:

1. you changed a canonical field name
2. you widened/narrowed a type without updating the fixture
3. you changed semantics without updating the contract layer first

Fix sequence:

1. check the frozen planning doc
2. check the shared contract type
3. check the fixture
4. check the consumer/provider test

Do not patch the frontend alone and ignore the canonical contract.

Problem: you are unsure whether something should become a new ADR

Ask:

1. Is this a real architecture or process decision?
2. Is it broader than one small implementation detail?
3. Would future readers need to know why this choice was made?

If yes, use ADR governance.

Current ADR process docs live in:

- `docs/adr/README.md`
- `docs/adr/_TEMPLATE.md`

Note:

- accepted ADRs should be numbered
- accepted ADRs should not be rewritten materially
- changed decisions should be superseded, not silently edited

17 Tips And Tricks

Tip 1: Read the planning bundle before expanding semantics

The planning bundle is not optional context. It is the definition of intended meaning.

Tip 2: Treat `packages/shared-types/` as sacred

If the frontend and backend disagree, fix it here first.

Tip 3: Keep slices narrow

The repo is intentionally growing by thin slices.

If you are touching:

- `run-state`
- `runtime summary`
- `signals`
- `approvals`

then keep the slice tightly bounded and verifiable.

Tip 4: Prefer fixtures early

Fixtures are useful in early slices because they:

- keep contracts concrete
- make DTO size checks easy
- let frontend and backend agree before deeper logic exists

Tip 5: Use the verify harness even when it is not strict

`PENDING` today becomes `PASS` or `FAIL` tomorrow.

The harness is part of how this repo avoids drifting back into a monolith.

Tip 6: Distinguish “bootstrap-real” from “feature-real”

Something can be real enough to run and still be intentionally shallow.

Current examples:

- runtime summary endpoint is real
- runtime summary assembly is still bootstrap-thin

That distinction matters during review.

Tip 7: Do not overreact to incomplete polish

At this stage, review should focus on:

- boundaries
- ownership
- contracts
- verification

Not whether the shell is already pretty or feature-rich.

18 What A Good Next Slice Looks Like

A good next slice should:

- keep shared contract semantics stable
- improve one owned behavior
- preserve boot simplicity
- come with verification

Completed thin slices (do not re-do):

- bootstrap + shared contracts + runtime summary

- first watch signal path
- npm workspace dev ergonomics
- first run lifecycle (create → executing → complete)
- startup supervision reliability (`scripts/dev/lib/common.sh`)
- stop/resume, approval, review-ready, SQLite persistence, and briefing-backed dock projections
- workspace list + backend PTY terminal + file-backed Monaco host + nested explorer tree + new-file creation + active-file rename + resizable bottom terminal dock
- richer inbox ranking (severity, recency, unresolved duration, status, action-type, workspace priority)
- split shell regions (`TopBar` , `LeftSidebar` , `CenterWorkbench` , `RightDock` , `StatusBar`) with mockup-shell HUD chrome
- Conversation and Command dock seams backed by control-plane chat endpoints (`POST /api/chat/messages` , `GET /api/workspaces/{workspace_id}/chat/thread` , `GET /api/chat/threads/{thread_id}/history`)

Suggested next slices (2026-07-04):

1. **Coordinator** — KAIRO operator-presence integration when explicitly assigned
2. **Lane B** — agent orchestration hook for chat messages (beyond system ack stub)

Bad next slices:

- broad UI rewrite
- expanding multiple semantic families at once
- changing run-state and signal-state in one uncontrolled pass
- skipping verification because “it is still early”
- claiming full IDE parity when workspace file operations are still intentionally thin-slice (open, edit, save, create, rename) rather than a full VS Code clone

19 Final Guidance

If you are unsure what to do next, choose the smaller move that:

- preserves ownership
- strengthens verification
- reduces placeholders
- keeps the shell boot-safe

That is the design center of this repo right now.